
CONSTRAINT SATISFACTION PROBLEMS

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

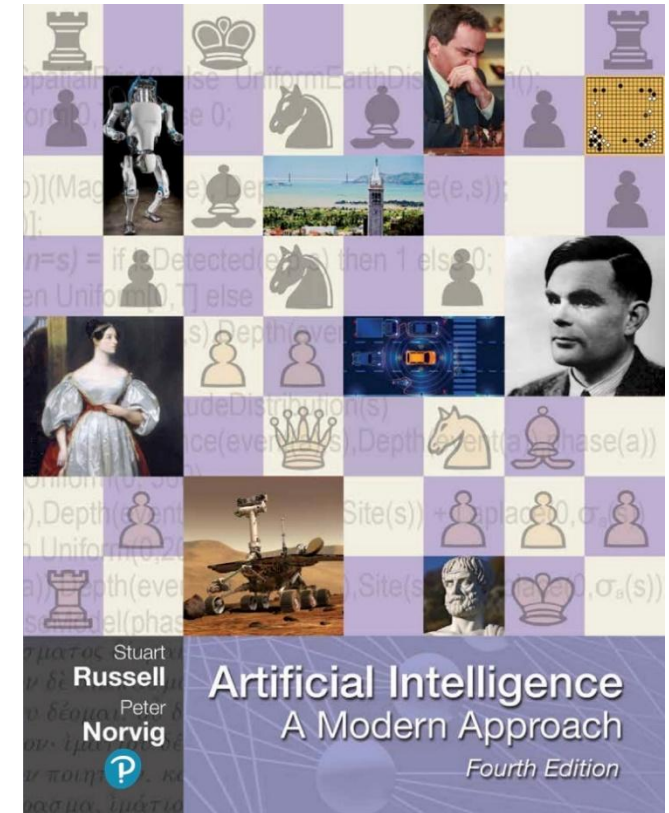
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy
- Some content adapted from Dr. Richard Lathrop (UC Irvine)

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 5 in AIMI 4/E



Outline

- What is a CSP?
- CSP Examples
- Solving CSPs:
 - Backtracking Search Algorithm
 - Backtracking + Heuristics
 - Backtracking + Inference (constraint-propagation) + Heuristics
- Exploiting Problem Structure and Decomposition
- Local Search for CSPs
- Summary

What is a CSP?

How CSP relates to standard search problems we have seen

- We discussed **path-searching problems** (uninformed and informed searching):
 - Solution is a sequences of actions from initial state to goal state.
 - The **path** to the goal is important.
 - Solution paths can have various costs and various depths in the search tree.
- We discussed **state-optimization problems** (local-search):
 - Solution is a configuration of the state.
 - The path is not important, the **goal state** itself is important.
 - In local search, we discussed algorithms for such problems.
- CSPs are a specialized class of state-optimization problems. In other words, they are another approach of tackling certain optimization problems.

Constraint satisfaction problems (CSP) – intro

- In CSPs, a state is a set of **variables**. The goal is to find an **assignment** of values to the variables that satisfies a set of **constraints**.
- Formally, a CSP consists of three components:
 - Set of variables $X = \{X_1, X_2, \dots, X_n\}$
 - Set of domains for each variable $D = \{D_1, D_2, \dots, D_n\}$. That is, $\forall_i. X_i \in D_i$
 - Set of constraints C . For example:
 - Implicit constraints using Boolean statements $C = \{X_1 < 2, X_3 > X_2, X_4 \neq X_1\}$
 - Explicit constraints using explicit values $C = \{(X_1, X_2) \in \{(1,2), (1,3), (2,3)\}, X_4 \in \{1,2,3\}\}$
- A **partial** assignment is one that assigns values to some variables, with some variables unassigned.
- A **complete** assignment assigns values to all variables.
- A **consistent** assignment is one that does not violate any constraints.
- A **complete and consistent** assignment is a **solution**.

Example: Map/Graph Coloring

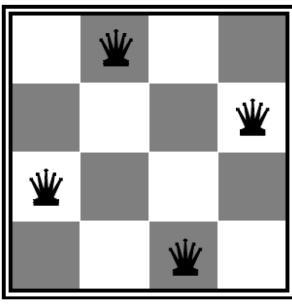
- Given a map of regions, the goal is to color each region such that no two neighboring regions have the same color.
- Variables: WA, NT, Q, NSW, V, SA, T
- Domains: $D = \{\text{red, green, blue}\}$
- Constraints: adjacent regions have different colors
 - Implicit: $WA \neq NT$
 - Explicit: $(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), \dots\}$
- Solutions are assignments satisfying all constraints, e.g.
 $\{WA=\text{red}, NT=\text{green}, Q=\text{red}, NSW=\text{green}, V=\text{red}, SA=\text{blue}, T=\text{green}\}$



Example: Job-shop scheduling

- Scheduling 15 tasks in car assembly: install axles (front/back), affix 4 wheels (right/left, front/back), tighten nuts (4 wheels), affix hubcaps (4 wheels), final inspection.
- Variables: $X = \{Axle_F, Axle_B, Wheel_{RF}, Wheel_{LF}, Wheel_{RB}, Wheel_{LB}, Nuts_{RF}, Nuts_{LF}, Nuts_{RB}, Nuts_{LB}, Cap_{RF}, Cap_{LF}, Cap_{RB}, Cap_{LB}, Inspect\}$
- Domain: time task starts. For examples, could be minute timestamps from 7AM to 7PM work hours.
- Constraints: specify precedence of tasks. For example, T_1 must occur before task T_2 , and T_1 takes d_1 minutes to complete. $T_1 + d_1 \leq T_2$
- Axles installed before wheels are affixed, and axle install takes 10 minutes:
$$Axle_F + 10 \leq Wheel_{RF}; \quad Axle_F + 10 \leq Wheel_{LF};$$
$$Axle_B + 10 \leq Wheel_{RB}; \quad Axle_B + 10 \leq Wheel_{LB}$$
- Can specify any Boolean constraint (e.g. disjunctive).
- See other constraints in textbook example. $(Axle_F + 10 \leq Axle_B) \quad \mathbf{or} \quad (Axle_B + 10 \leq Axle_F)$

Example: N -queens [1]



- Given an $N \times N$ grid, position n queens such that no queen attacks another.
- Formulation 1
 - Variables: $N \times N$ variables, one for each grid position (i, j) . X_{ij} for $(i, j) \in \{1, 2, \dots, N\} \times \{1, 2, \dots, N\}$
 - Domain: $X_{ij} \in \{0, 1\}$ where 0 denotes no queen, 1 denotes queen present.
 - Constraints:

Row constraint: $\forall i, j, k \ (X_{ij}, X_{ik}) \in \{(0, 0), (0, 1), (1, 0)\}$

Column constraint: $\forall i, j, k \ (X_{ij}, X_{kj}) \in \{(0, 0), (0, 1), (1, 0)\}$

lower-right diagonal: $\forall i, j, k \ (X_{ij}, X_{i+k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$

lower-left diagonal: $\forall i, j, k \ (X_{ij}, X_{i+k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$

upper-right diagonal: $\forall i, j, k \ (X_{ij}, X_{i-k, j+k}) \in \{(0, 0), (0, 1), (1, 0)\}$

upper-left diagonal: $\forall i, j, k \ (X_{ij}, X_{i-k, j-k}) \in \{(0, 0), (0, 1), (1, 0)\}$

N queens placed: $\sum_{i,j} X_{ij} = N$

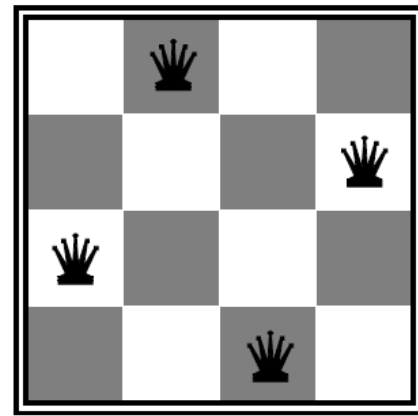
Example: N -queens [2]

- Given an $N \times N$ grid, position n queens such that no queen attacks another.
- Formulation 2
 - Variables: N variables, one for each column. Denoted as Q_i for $i \in \{1, 2, \dots, N\}$
 - Domain: $\forall i. Q_i \in \{1, 2, \dots, N\}$ denoting row position of the queen.
 - Constraints:

Implicit: $\forall i, j \quad \text{non-attacking}(Q_i, Q_j)$

Explicit: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \dots\}$
...

- Reduced state space from $N \times N$ to just N . Less states to search through!



Example: Boolean Satisfiability (SAT)

- Given a Boolean formula, find an assignment of True/False to the variables that satisfies the entire formula (makes the formula evaluate to True)

$$x_1 \wedge \neg x_2 \vee \neg x_4 \wedge x_3 \wedge \dots \vee x_n$$

- Formulation

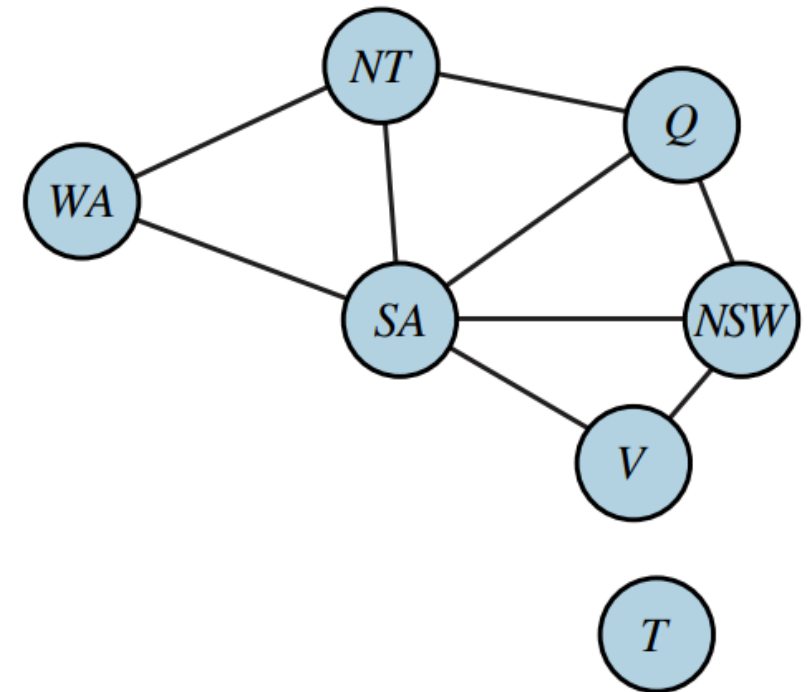
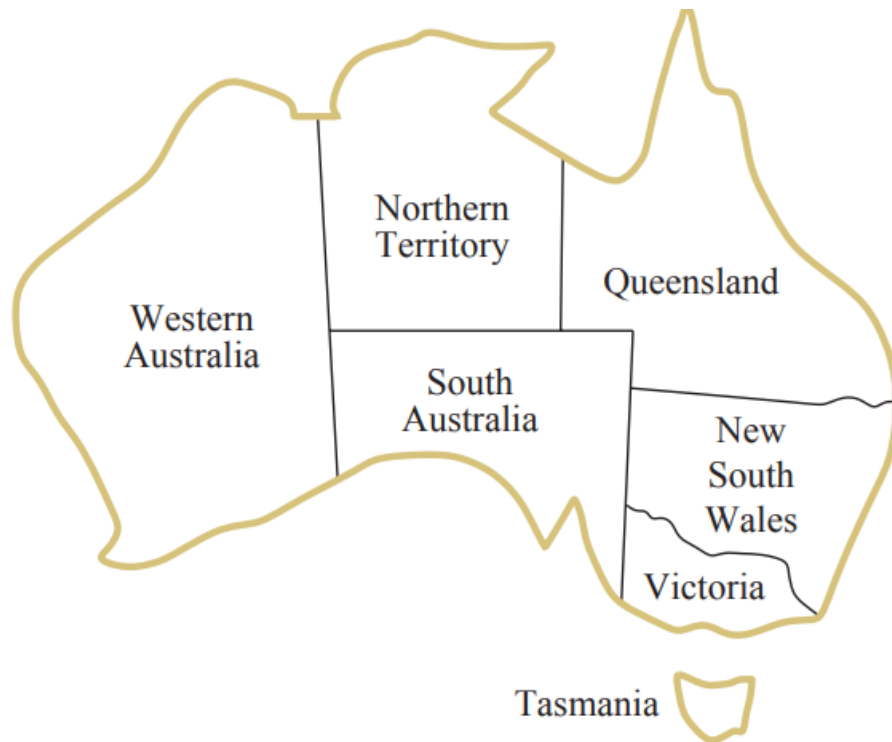
- Variables: n variables, denoted as x_i for $i \in \{1, 2, \dots, N\}$
- Domain: $\forall i. x_i \in \{T, F\}$ denoting true/false assignment.
- Constraints:
 - Can use the entire Boolean formula as a single constraint.
 - Can break down the formula to sub-formulas based on the variables involved in the sub-formulas.
 - Other encodings possible that is particular to SAT (convert to 3-CNFSAT and encode clause constraints).
- SAT is an NP-Complete problem. No known efficient algorithm for all cases. Worst-case we have to check all possible assignments $O(2^n)$
- However, in practice, many formulas can be solved quickly using heuristic-based solvers (DPLL is a popular solver).

Examples of other applications

- Assignment problems , e.g., who teaches what? Constraints can be teaching/time preference.
- Scheduling problems, e.g. transportation schedules, machine/job factory scheduling, etc.
- Timetabling problems, e.g., which class is offered when and where?
- Graph/map coloring, vertex labelling, networking problems.
- Hardware configuration.
- Floor planning and circuit layout.
- Cryptography, computer vision, fault diagnosis, etc.

Using constraint graph to visualize CSPs

- After specifying the variables, domains, and constraints, we can visualize the CSP as a constraint graph.
- Constraint graph: **nodes** represent **variables**, **arcs/edges** represent **constraints**.
- The CSP graph structure gives us insight in how to speed up search. E.g., Tasmania is an independent subproblem (more on this later).



Varieties of CSPs: Variables

- Finite domain Variables
 - With domain of size d for all variables, how many complete assignments? $O(d^n)$
 - We will focus on just these type of CSPs in this course.
- Infinite domain variables
 - E.g., start/end times for tasks that require precise timing (real-values in some interval)
 - Depending on the type of constraints (linear, non-linear) can use solver techniques from the field of optimization.
 - Can *discretize* the domain, turning it into finite-domain variables.

Varieties of CSPs: Constraints

- **Unary** constraints involve a single variable (equivalent to reducing domains), e.g.: $SA \neq green$
- **Binary** constraints involve pairs of variables, e.g.: $SA \neq WA$
- **Higher-order** constraints involving 3 or more variables:
 - In a constraint graph, can be represented by **hyper-nodes**
 - E.g. implicit constraint with three variables $A + B < 3C$
 - Can always be represented as binary constraints by introducing auxiliary variables. See [link](#).
 - **Global constraint**: has arbitrary number of variables. E.g. *AllDiff* constraint saying that all variables involved must have different values.
- Preferences (soft constraints):
 - E.g., red is better than green.
 - Optimization of a cost function with constraints (constrained optimization).
 - Can return all possible CSP solutions and then calculate their cost. Select one with min cost.
- Typical CSP solvers (packages) operate on binary constraints. You can convert n -ary constraints in a CSP to binary constraints by introducing auxiliary variables. In practice, there are packages/frameworks that can do this for you. In this course, we will operate on binary CSPs.

Example Sudoku [1]

- In sudoku, goal is to place digits to complete the puzzle with constraints on digit placement.

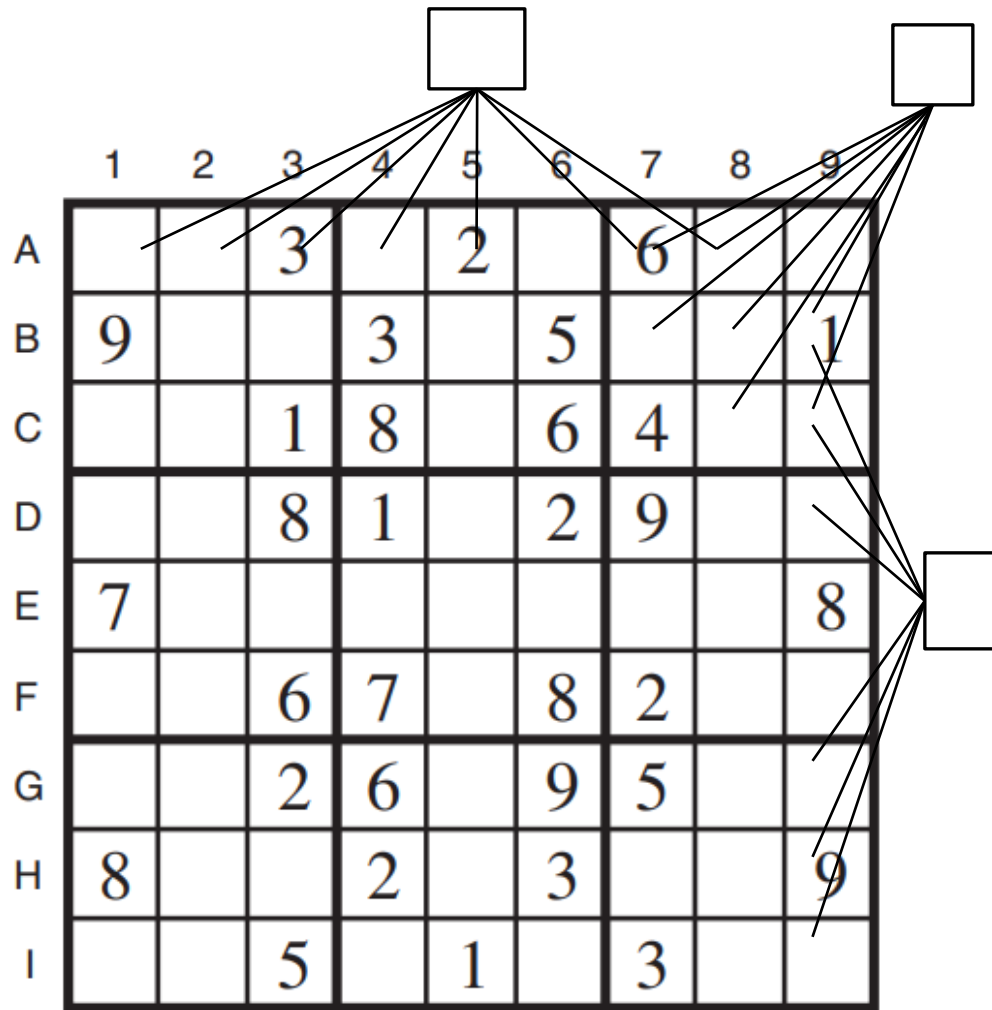
	1	2	3	4	5	6	7	8	9
A			3		2		6		
B	9			3		5			1
C			1	8		6	4		
D			8	1		2	9		
E	7								8
F			6	7		8	2		
G			2	6		9	5		
H	8			2		3			9
I			5		1		3		

Initial state

	1	2	3	4	5	6	7	8	9
A	4	8	3	9	2	1	6	5	7
B	9	6	7	3	4	5	8	2	1
C	2	5	1	8	7	6	4	9	3
D	5	4	8	1	3	2	9	7	6
E	7	2	9	5	6	4	1	3	8
F	1	3	6	7	9	8	2	4	5
G	3	7	2	6	8	9	5	1	4
H	8	1	4	2	5	3	7	6	9
I	6	9	5	4	1	7	3	8	2

Solution

Example Sudoku [2]



- Variables: Each (open) square
- Domains: $\{1, 2, \dots, 9\}$
- Constraints:
 - 9-way alldiff for each column
 - 9-way alldiff for each row
 - 9-way alldiff for each 3x3 region

or can have a bunch of binary inequality constraints: $A1 \neq A2$, $A2 \neq A3$, ...

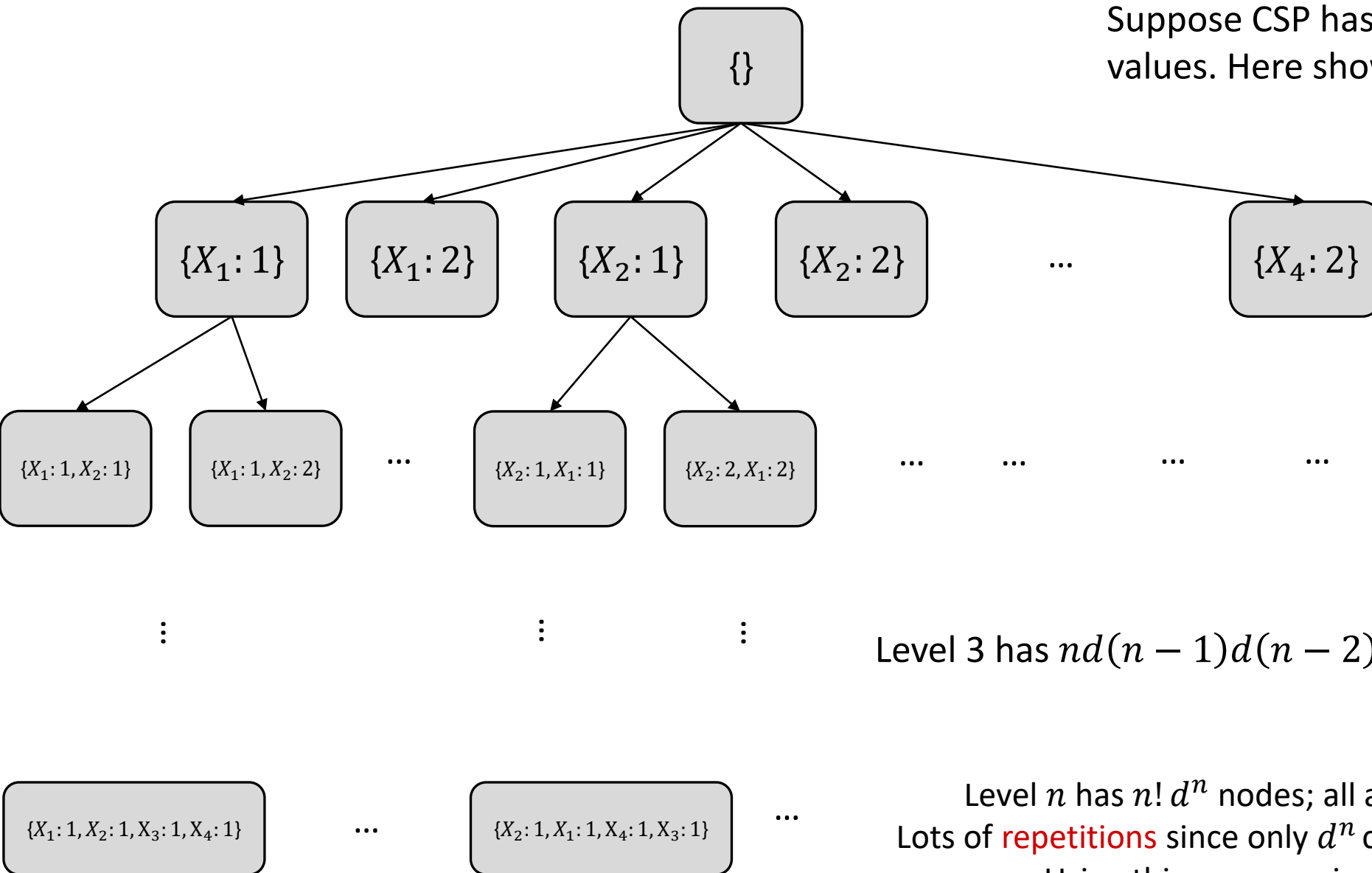
Solving CSPs – Backtracking Search

CSP as a standard search formulation [1]

- We can solve CSPs using standard search algorithms like BFS or DFS.
- Define the CSP as a standard search problem.
- Use an *incremental* state formulation
 - States: *partial* assignment of variables.
 - Initial State: empty assignment {}
 - Actions (successor function): assign value to any unassigned variable such that no conflict with current assignment.
 - Goal test: current assignment is **complete** and **consistent**.
 - Path cost: constant cost for every step (not really relevant)

CSP as a standard search formulation [2]

Suppose CSP has n variables, each with d values. Here shown $n = 4$ and $d = \{1, 2\}$



Level 1 has nd nodes

Level 2 has $nd(n - 1)d$ nodes; notice we have repeated states showing up

Level 3 has $nd(n - 1)d(n - 2)d = n(n - 1)(n - 2)d^3$ nodes

Level n has $n! d^n$ nodes; all are complete assignments.
Lots of **repetitions** since only d^n complete assignments possible.
Using this process, size of full tree $O(n! d^n)$

CSP as a standard search formulation [3]

- Assume there are n variables, each with d possible values.
- How can we use a search algorithm like BFS to solve this?
- Depth of complete assignments (the leaves of the search tree)?
 - n assigned variables, so leaves at depth n .
- How many possible complete assignments?
 - d^n
- If we apply standard breadth-first search:
 - Branching factor at root?
 - nd (choose among n variables, then choose a value for that variable)
 - Branching factor at level 1?
 - $(n - 1)d$ (choose among $n - 1$ variables, then choose value)
 - Thus, # nodes at depth n (# leaves at last depth of search tree)?
 - $n! d^n$
- Can we exploit a CSP property to reduce # of nodes?

Commutativity of CSPs

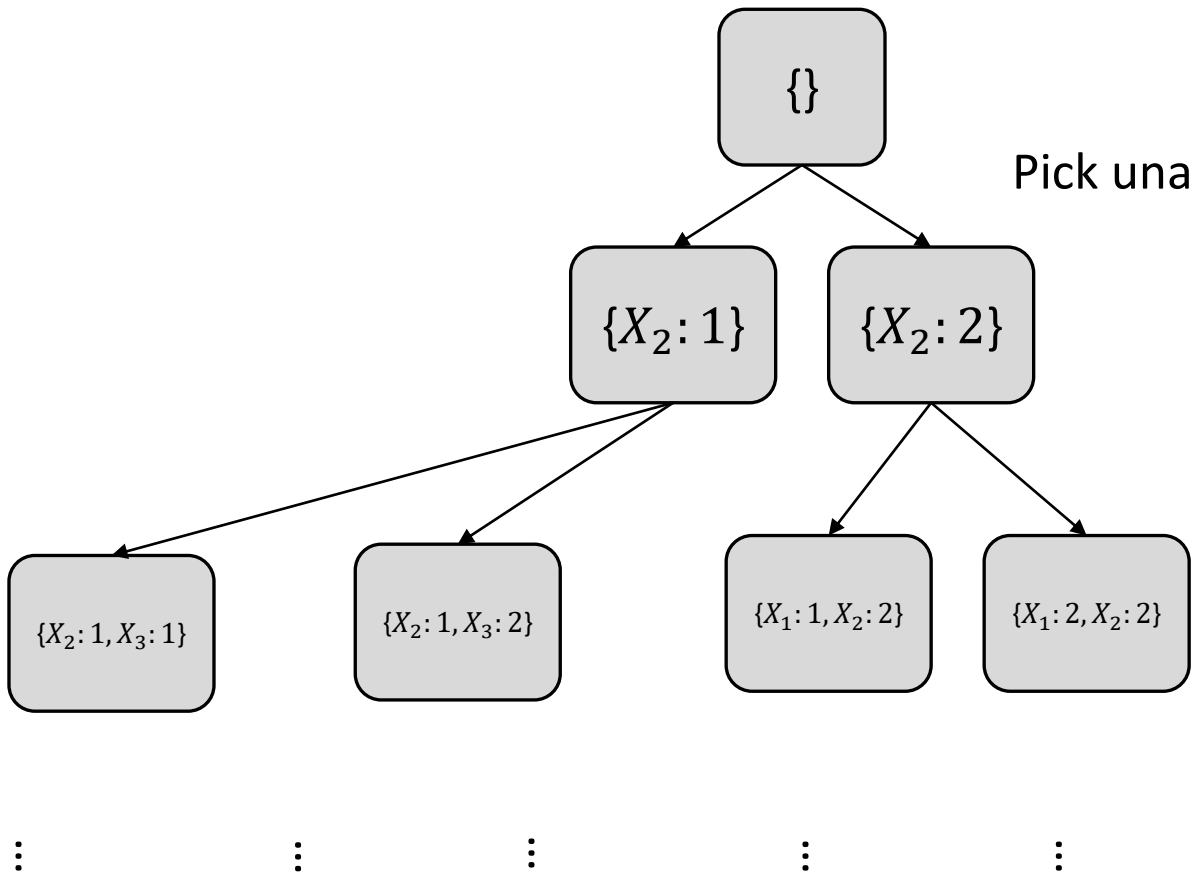
- In CSPs, variable assignments (actions) are commutative
 - order in which variables are assigned has no effect on outcome. Path to goal is irrelevant, only getting goal is important.
 - E.g.: choosing colors for Australian territories [WA=red then NT=green] same meaning as [NT=green then WA=red]
- CSP search algorithms can generate successors for a node by considering assignments for only a single variable.
- When generating children for a node in the search tree, we first select a variable using some criteria (more on this later). Once a variable is selected at a node, it will have d children.
- By exploiting this commutativity, there are d^n leaves for the search tree.
- This gives rise to a simple CSP search algorithm: **backtracking search**.

Backtracking search

- In CPSs, variable assignments (actions) are commutative. Order in which variables are assigned has no effect on outcome. Path to goal is irrelevant, only getting goal is important.
 - E.g. choosing colors for Australian territories [WA=red then NT=green] same meaning as [NT=green then WA=red]
- Backtracking search exploits this commutative property. Its behavior is similar to tree-like DFS.
- Idea 1: One variable at a time
 - At each node, we pick a single variable to assign values to. This gives d children for each node.
- Idea 2: Check constraints as you go
 - Consider only values which do not conflict with previous assignments
 - Might have to do some computation to check the constraints
- If value assigned to variable violates constraint, then try other values. If all values for that variable violate some constraint, then **backtrack** to previous node in the DFS path and try other paths.
- Continue DFS until solution is found or we backtrack to initial variable (root), in which case no possible solution.
- Worst case time complexity is $O(d^n)$
- What about space complexity?

CSP using backtracking (exploiting commutativity)

Suppose CSP has n variables, each with d values. Here shown $n = 4$ and $d = \{1, 2\}$

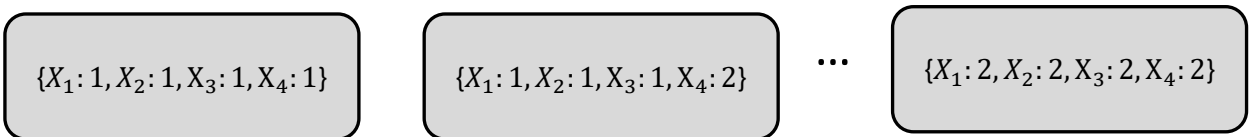


Pick unassigned variable to assign to the children of a node

Level 1 has d nodes

Level 2 has d^2 nodes; previous level has d each with d children; notice no more repetition

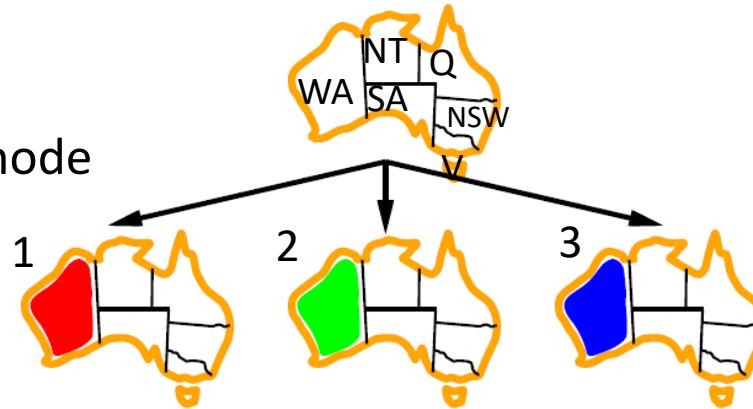
Level 3 has d^3 nodes; previous level has d^2 each with d children
⋮



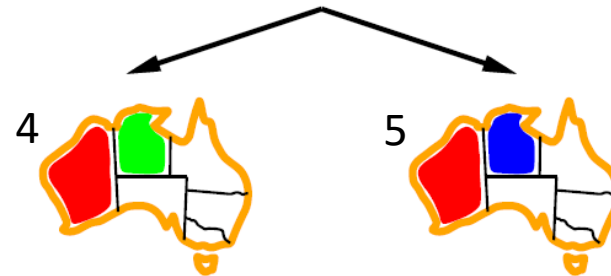
Level n has d^n nodes; all are complete assignments.
No repetition. Path not important; just assignment.
Worst-case time $O(d^n)$

Map coloring example

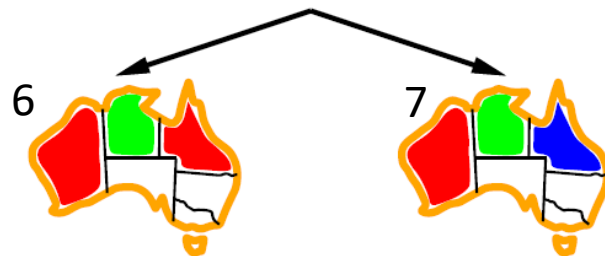
Pick WA to assign to children of root node



Pick NT to assign to children of node 1



Pick Q to assign to children of node 4



Note: a *criteria* is used to select which variable to assign to the children of **each node (NOT at each level)**.

For example, node 2 could have selected variable NSW for its children; instead of node 1 which selected NT.

```

function BACKTRACKING-SEARCH(csp) returns solution/failure
    return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment given CONSTRAINTS[csp] then
            add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then return result
            remove {var = value} from assignment
    return failure

```

- Backtracking = DFS + **variable-and-value-ordering** + **fail-on-violation**
- How to define variable ordering, value ordering, constraint propagation?
- For this implementation, how many stack frames are possible? Since we do not generate all children of a node at same time, only one child is generated at any given time, so max recursion stack frames at any point is $O(n)$.

Improving backtracking

- Recall that uninformed search algorithms can be improved by using **domain-specific (problem-specific)** heuristics. This gave us informed search like A^* .
- By taking advantage of factored/partial state representation in CSPs, backtracking search can be improved by using **domain-independent** heuristics. Making it a powerful general-purpose algorithm.
- How can we possibly speed-up backtracking search:
 - Ordering:
 - Which variable should be assigned next?
 - In what order should its values be tried?
 - Filtering/Inference: Can we use consistency-checking to reduce search space? Can we use consistency-checking to detect failure early?
 - Structure: Can we exploit the problem/constraint-graph structure?

Filtering/Inference: Node consistency **before** searching

- Typical CSP solvers operate on binary constraints. As we mentioned, you can convert n -ary constraints in a CSP to binary constraints by introducing auxiliary variables.
- What about unary constraints? Unary constraints restrict the domains of a variable. So, we can enforce this restriction before running backtracking search.
- A single variable is **node-consistent** if all the values in the variable's domain satisfy the variable's unary constraints.
- So, given a CSP problem with unary and binary constraint, we can enforce node consistency by checking the unary constraints and reducing the domains appropriately. The CSP becomes a binary CSP and we can run backtracking search.

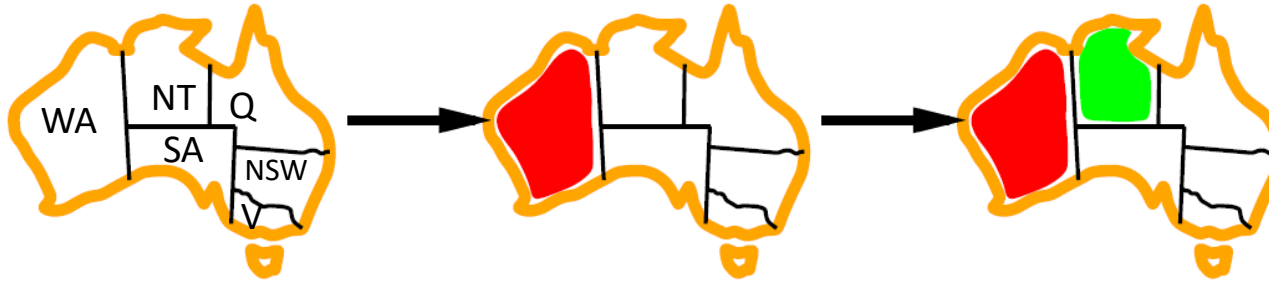
Improving Backtracking Search – Variable and Value Ordering

Variable ordering

- When at a current node, we need to select an *unassigned* variable. The children of the current node will be the different values assigned to the selected variable.
- Idea: choose variables that are highly constrained early in the search process; leave easy ones for later.
- **Minimum Remaining Values (MRV)**
 - Choose the unassigned variable with the fewest remaining legal values.
 - Also called “**most constrained variable**” or “**fail-first**” heuristic because it picks a variable that is likely to fail soon.
- **Degree heuristic**
 - Select variable involved in largest number of constraints on other *unassigned* variables.
 - Can be used as a tie breaker among variables with same MRV.
- An idea is to use MRV for variable selection, and degree for tie-breaking.

MRV Example

- MRV: choose the unassigned variable with the fewest remaining legal values



At this current node.

SA: 1 remaining legal color

Q: 2 remaining legal colors

NSW, V, and T: 3 remaining legal colors

Choose SA

Notes on MRV usage



At this current node.

SA: 1 remaining legal color

Q: 2 remaining legal colors

NSW, V, and T: 3 remaining legal colors

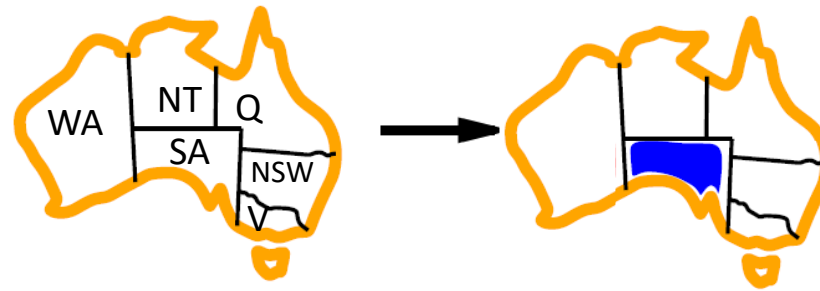
Choose SA

- For our implementation (python). We pair MRV with a filtering technique like forward-checking or AC3.
- Why? Because our MRV implementation just counts domain sizes; it does not filter down variable domains.
- So, the workflow inside a node is: spick a variable using MRV, order values, assign selected variable a value, apply filtering using FC or AC3 to reduce domains, then recursive call on child node. The child node will have reduced domains due the parent's filtering, and its MRV call will have meaningful usage.
- If we do not filter domains and call MRV (counting domains), then unassigned variables will have same domain sizes.
- So, if we use MRV, we pair it with filtering.

Degree Example

- Degree: select variable involved in largest number of constraints on other unassigned variables
- Use as tie-breaker.

At root node, all have same MRV
Use degree to break tie.
SA has most constraints with other
unassigned variables

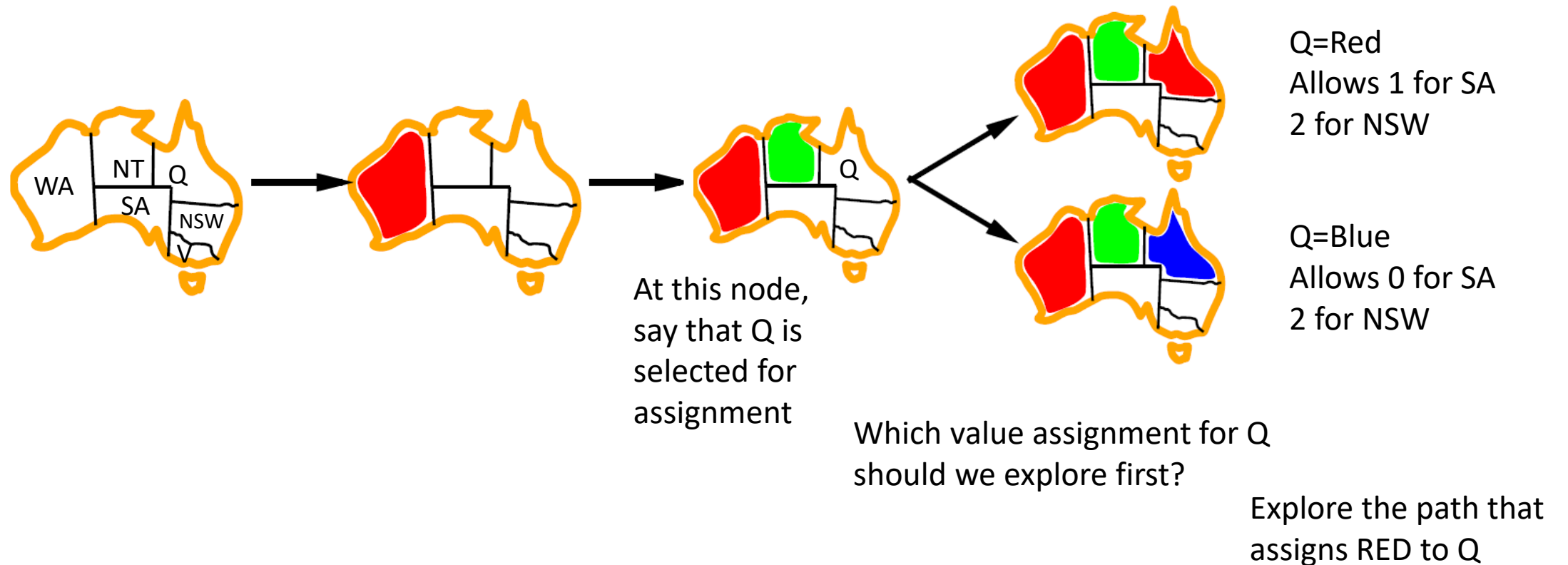


Value ordering

- Once unassigned variable is selected, decide order of values for that variable.
- **Least-constraining-value (LCV)**
 - Choose the value that rules out the fewest choices for neighboring unassigned variables
 - Attempts to give highest flexibility to subsequent variable assignments
 - Orders the values of a variable from least-constraining to most-constraining.
- Why variable ordering with MRV is fail-first, but value-ordering with LCV is fail-last?
 - All variables must be assigned a value. By going fail-first, we can prune large invalid parts of the search tree early.
 - For value-ordering, we only need one solution, so we want to give flexibility to remaining variables.
 - If these arguments are not convincing, that is a valid opinion. These orderings are **heuristics**, and not **guaranteed** to speed up solution. Although they work well in practice for a variety of problems.

LCV Example

- Least-constraining-value (LCV): choose the value that rules out the fewest choices for neighboring *unassigned* variables.



```

function BACKTRACKING-SEARCH(csp) returns solution/failure
    return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment given CONSTRAINTS[csp] then
            add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then return result
            remove {var = value} from assignment
    return failure

```

- Backtracking = DFS + **variable-and-value-ordering** + **fail-on-violation**

Improving Backtracking Search – Filtering/Inference

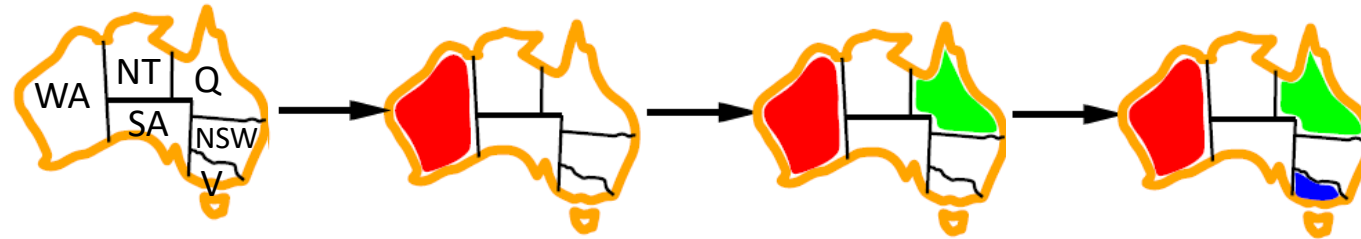
Filtering/Inference: Node consistency **before** searching

- Typical CSP solvers operate on binary constraints. As we mentioned, you can convert n-ary constraints in a CSP to binary constraints by introducing auxiliary variables.
- What about unary constraints? Unary constraints restrict the domains of a variable. So, we can enforce this restriction before running backtracking search.
- A single variable is **node-consistent** if all the values in the variable's domain satisfy the variable's unary constraints.
- So, given a CSP problem with unary and binary constraint, we can enforce node consistency by checking the unary constraints and reducing the domains appropriately. The CSP becomes a binary CSP and we can run backtracking search.

Filtering/Inference: Simple Forward-checking **during** search

- When a variable is assigned a value, there is an opportunity to filter/infer new reductions on **domains** of unassigned variables.
- Can detect failure early to avoid going down that path in search tree, potentially speeding up the search
- Simplest filtering method is **forward checking**:
 - In each search node, keep track of remaining legal values for unassigned variables. This can be done by maintaining a domain dictionary `current_domains = {var_name: domain_list_of_var}`.
 - After variable X is assigned value x , check each unassigned neighbor of X , and delete from that neighbor variable's domain any value that is inconsistent with value $X=x$.
 - Backtrack when an unassigned variable has no legal values; i.e. `current_domains[v]` of unassigned variable becomes empty.

Forward-checking example



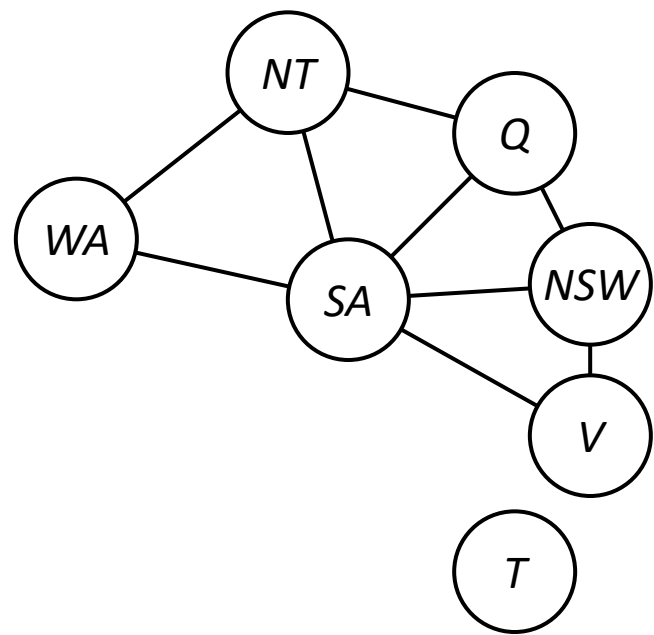
	<i>WA</i>	<i>NT</i>	<i>Q</i>	<i>NSW</i>	<i>V</i>	<i>SA</i>	<i>T</i>	
Initial domains								
After <i>WA=red</i>								
After <i>Q=green</i>								
After <i>V=blue</i>								

SA has no more values left to assign. FC has detected that partial assignment is inconsistent with the constraints and backtracking can occur now.

Notice that *each* node in the CSP search tree updates the variable domains.

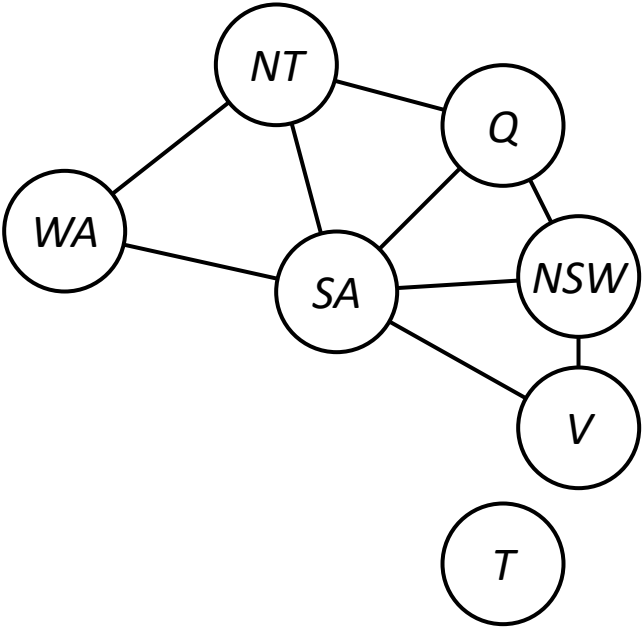
Example with FC with fixed variable order

	Region (Variable)	Domain		
→	<i>T</i>			
→	<i>NT</i>			
→	<i>V</i>			
→	<i>SA</i>			
→	<i>NSW</i>			
→	<i>Q</i>			
→	<i>WA</i>			



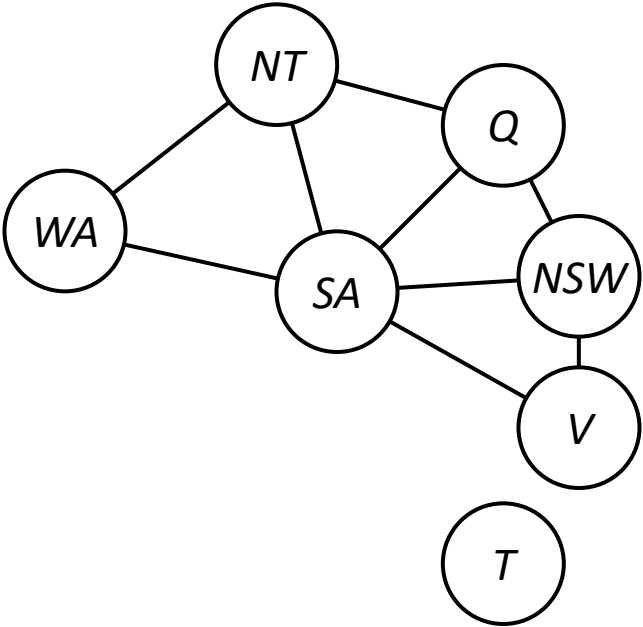
Example with FC + MRV

Region (Variable)	Domain		
T			
NT			
V			
SA			
NSW			
Q			
WA			



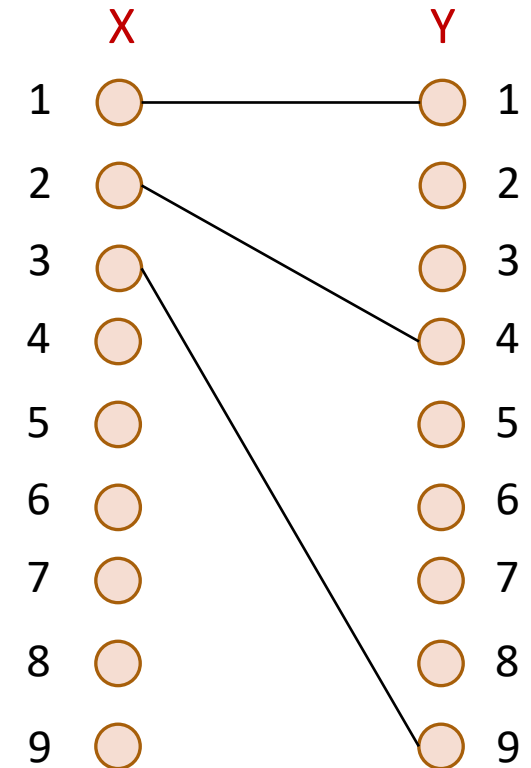
Example with FC + MRV + Degree

Region (Variable)	Domain		
T			
NT			
V			
SA			
NSW			
Q			
WA			



Constraint Propagation: Arc-consistency **before** search

- A stronger notion than node-consistency is arc-consistency. It enforces binary constraints.
- Formally:
An arc $X \rightarrow Y$ is **consistent** iff for every value of X in current domain of X , there is *some* value of Y in current domain of Y , such that the constraint between X and Y is consistent (i.e. not violated).
- A CSP is **arc-consistent** if every variable is arc consistent with every other variable.
- Example: consider the simple CSP with two variables X and Y :
 - Domain of both variables: $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 - Constraint: $Y = X^2$ or in other words $\langle X, Y \rangle \in \{(1,1), (2,4), (3,9)\}$
 - To make $X \rightarrow Y$ consistent, then we make $\text{domain}(X) = \{1, 2, 3\}$
 - To make $Y \rightarrow X$ consistent, then we make $\text{domain}(Y) = \{1, 4, 9\}$
 - With these **updated** domains, the CSP is arc-consistent.
- Given a CSP, we can enforce arc consistency on all arcs to reduce domains, then we can run backtracking search.



AC-3 Algorithm

function AC-3(*csp*) **returns** false if an inconsistency is found and true otherwise
queue $\leftarrow$ a queue of arcs, initially all the arcs in *csp*

while *queue* is not empty **do** */* initial queue must contain both (X_i, X_j) and (X_j, X_i) */*

$(X_i, X_j) \leftarrow \text{POP}(\text{queue})$

if REVISE(*csp*, X_i , X_j) **then**

if size of $D_i = 0$ **then return** false

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

 add (X_k, X_i) to *queue*

return true

function REVISE(*csp*, X_i , X_j) **returns** true iff we revise the domain of X_i

revised $\leftarrow$ false

for each x **in** D_i **do**

if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j **then**

 delete x from D_i

revised $\leftarrow$ true

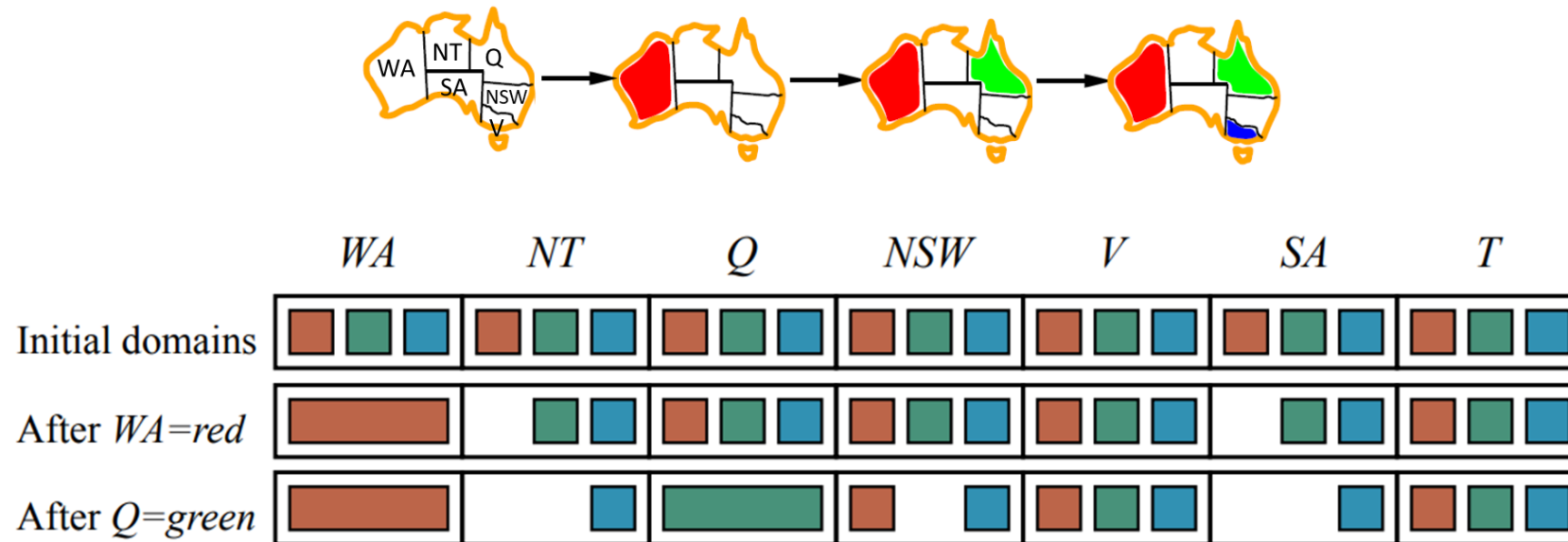
return *revised*

- AC-3 algorithm that can be run on a binary CSP prior to backtracking search.
- **Initially**, the queue (implement as a set) contains all the arcs in the CSP **in both directions**. So, a binary constraint is two arcs.
- AC-3 then pops an arc (X_i, X_j) from the queue and makes X_i arc-consistent with respect to X_j
- If the *domain*(X_i) is reduced, then we add to the queue all arcs (X_k, X_i) where X_k is a neighbor of X_i
- Once the queue is empty, then the CSP is arc-consistent.
- What if the domain of a variable becomes empty?

- With n variables and d max domain size. At most $O(n^2)$ total arcs. Each arc can be inserted into queue at most d times. Checking consistency of a single arc is d^2 .
- Total time-complexity: $O(n^2d^3)$. Other variant not discussed in course called AC-4 with $O(n^2d^2)$.

Arc-consistency **during** search

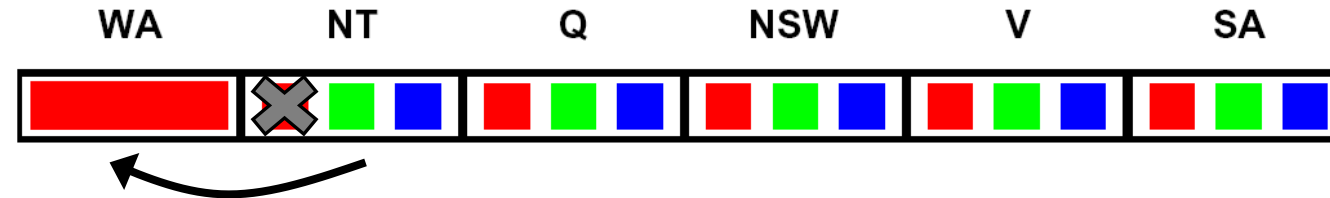
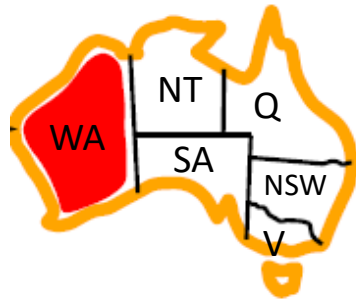
- FC propagates information from **assigned** to **unassigned** variables.
- Solving CSPs with combination of heuristics + FC can *potentially* reduce search work.
- But FC doesn't provide early detection for *all* future failures.



- After assigning $Q=green$, NT and SA cannot both be blue. Detecting failure early here would be better.
- We can run AC-3 **during** backtracking search to detect these kind of failures.

Arc-consistency **during** search

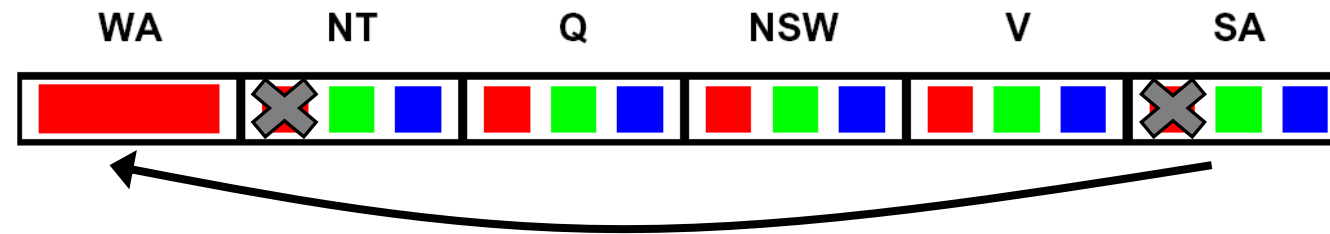
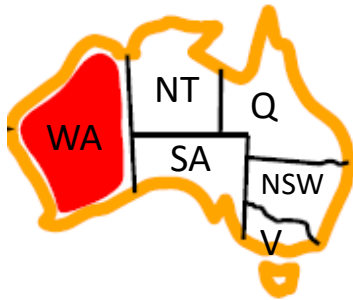
- An arc $X \rightarrow Y$ is **consistent** iff for every value of X in current domain of X , there is some value of Y in current domain of Y , such that the constraint between X and Y is consistent (i.e. not violated).
- After assigning a value to a variable X , we can run AC-3 with $queue = \{(Y, X) \text{ where } Y \in neighbors(X)\}$. In other words, we enforce consistency on arcs $Y \rightarrow X$ for each **unassigned** Y that is a neighbor of X .
- Example: after assigning $WA = \text{Red}$, we can run AC-3 starting with $queue = \{NT \rightarrow WA, SA \rightarrow WA\}$
- **Important:** Recall that in AC-3, if X loses a value, all neighbors of X need to be rechecked (added to queue).
- Pop an arc from $queue = \{NT \rightarrow WA, SA \rightarrow WA\}$. Say we pop $NT \rightarrow WA$ and enforce arc-consistency:



- Since $domain(NT)$ was reduced, we add $Q \rightarrow NT$ and $SA \rightarrow NT$ to the queue.

Arc-consistency **during** search

- The queue currently contains $\{SA \rightarrow WA, Q \rightarrow NT, SA \rightarrow NT\}$
- Let's say we pop $SA \rightarrow WA$

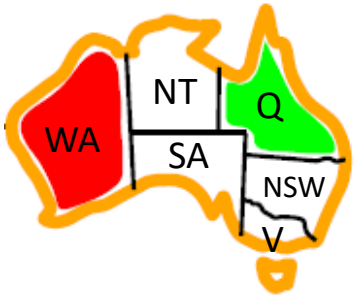


- Since $\text{domain}(SA)$ was reduced, then $\text{queue} = \{Q \rightarrow NT, SA \rightarrow NT, NT \rightarrow SA, Q \rightarrow SA, NSW \rightarrow SA, V \rightarrow SA\}$
- AC-3 will pop the remaining arcs in the queue and check them. These remaining arcs are arc-consistent, so all without modifying any domain. No issues so far as no domain is empty, so searching continues.
- Let's say we assign $Q = \text{Green}$ next.



Arc-consistency **during** search

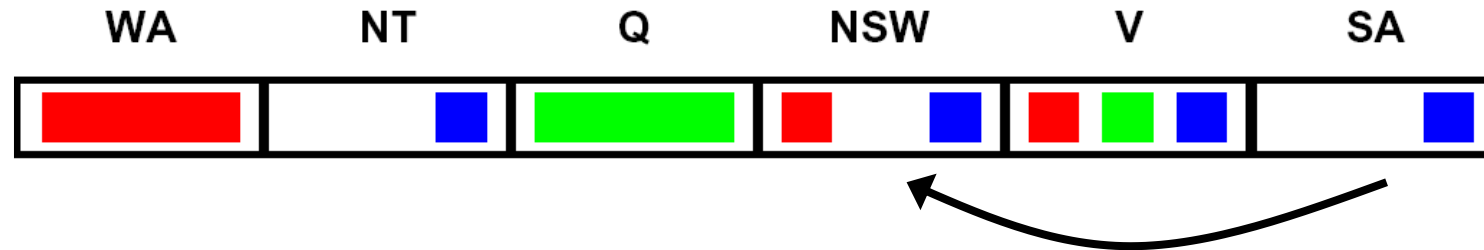
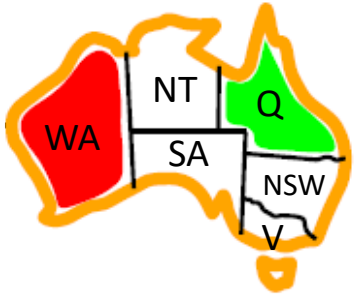
- Q was assigned Green, so we run AC-3 queue = {NT $\rightarrow$ Q, SA $\rightarrow$ Q, NSW $\rightarrow$ Q}. We pop these arcs and check them. This will **remove** Green from the domains of NT, SA, and NSW.
- After adjusting these domains, queue = {SA $\rightarrow$ NT, WA $\rightarrow$ NT, WA $\rightarrow$ SA, NT $\rightarrow$ SA, NSW $\rightarrow$ SA, V $\rightarrow$ SA, SA $\rightarrow$ NSW, V $\rightarrow$ NSW}. We are adding **unassigned** neighbors of modified domains.
- Note, we did not add Q $\rightarrow$ NT since Q was just assigned. We also do not need to add WA $\rightarrow$ NT and WA $\rightarrow$ SA even though the domains of NT and SA were reduced (why?)



Pop V $\rightarrow$ NSW. No domain changes.

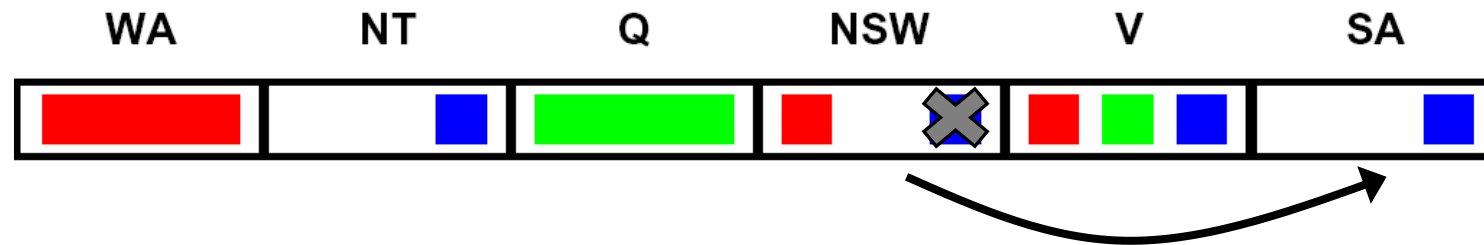
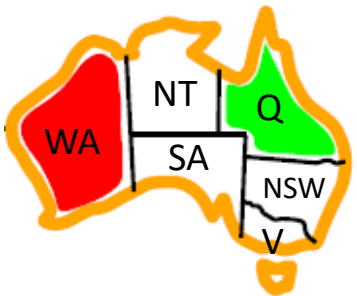
queue = {SA $\rightarrow$ NT, NT $\rightarrow$ SA, NSW $\rightarrow$ SA, V $\rightarrow$ SA, SA $\rightarrow$ NSW}

Arc-consistency **during** search



Pop SA $\rightarrow$ NSW. No domain changes.

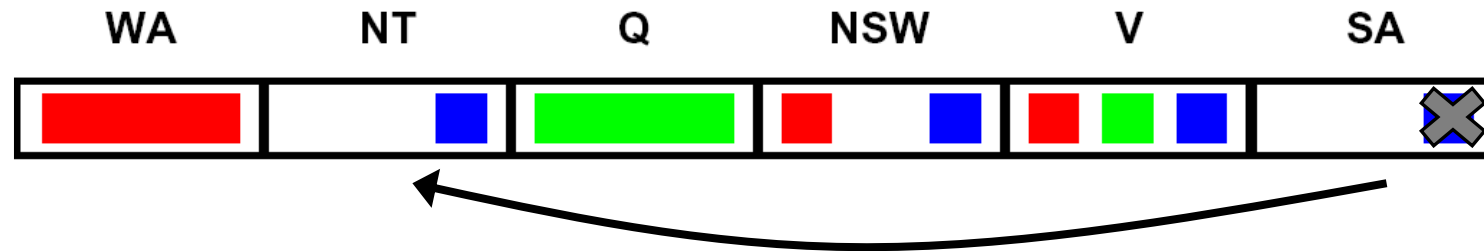
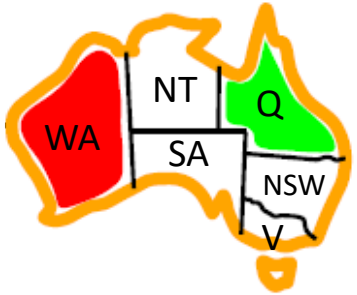
queue = {SA $\rightarrow$ NT, NT $\rightarrow$ SA, NSW $\rightarrow$ SA, V $\rightarrow$ SA}



Pop NSW $\rightarrow$ SA. Need to add V $\rightarrow$ NSW, SA $\rightarrow$ NSW. If arc already exists in queue do not add it. Also, no need to add Q $\rightarrow$ NSW since Q has already been assigned.

queue = {SA $\rightarrow$ NT, NT $\rightarrow$ SA, V $\rightarrow$ SA, V $\rightarrow$ NSW, SA $\rightarrow$ NSW}

Arc-consistency **during** search



Pop SA → NT. Inconsistency detected. We can stop AC-3 and go back to backtrack search.

- Arc-consistency can detect some failures earlier than FC.
 - However, note that requires more computation.
 - What is time-complexity of FC, what is time-complexity of AC-3?
 - This check is done in *each* search node. After assigning a value to variable, apply AC3 on current domains.
- Note that arc pop order also matters in reducing computation in case of failure cases.
- Notice that enforcing arc-consistency is strictly more powerful than forward checking because forward checking does the same thing on the initial arcs, but FC does not recursively propagate constraints when changes are made to the domains of variables.

Notes on MRV usage



At this current node.

SA: 1 remaining legal color

Q: 2 remaining legal colors

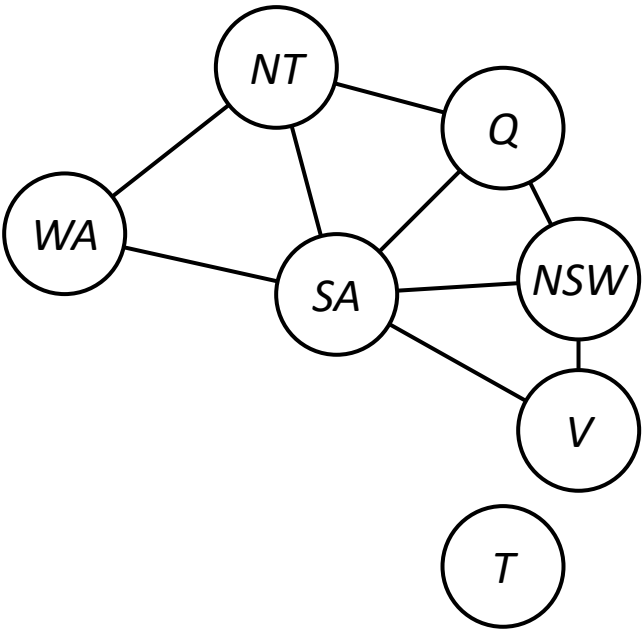
NSW, V, and T: 3 remaining legal colors

Choose SA

- For our implementation (python). We pair MRV with a filtering technique like forward-checking or AC3.
- Why? Because our MRV implementation just counts domain sizes; it does not filter down variable domains.
- So, the workflow inside a node is: spick a variable using MRV, order values, assign selected variable a value, apply filtering using FC or AC3 to reduce domains, then recursive call on child node. The child node will have reduced domains due the parent's filtering, and its MRV call will have meaningful usage.
- If we do not filter domains and call MRV (counting domains), then unassigned variables will have same domain sizes.
- So, if we use MRV, we pair it with filtering.

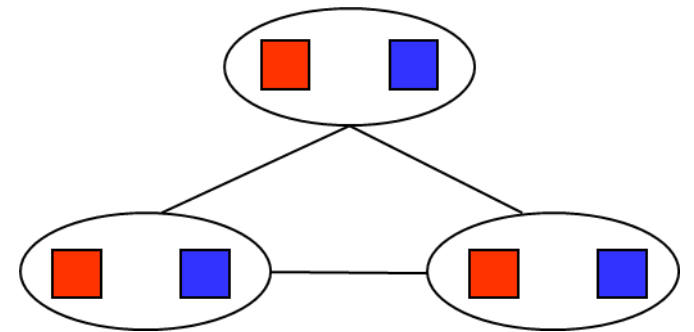
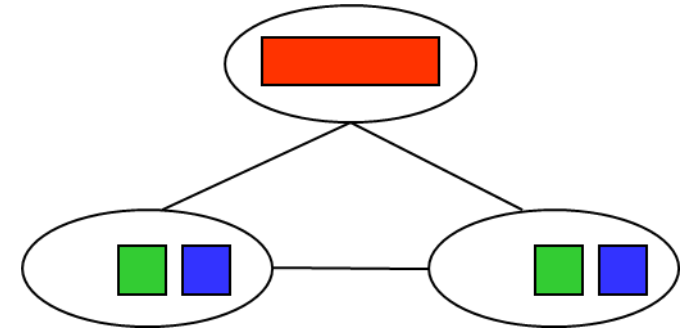
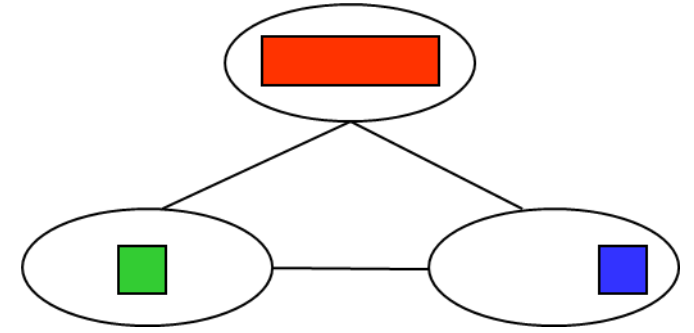
Example with AC

	Region (Variable)	Domain		
→	SA	●	●	●
→	NT	●	●	●
→	V	●	●	●
→	T	●	●	●
→	NSW	●	●	●
→	Q	●	●	●
→	WA	●	●	●



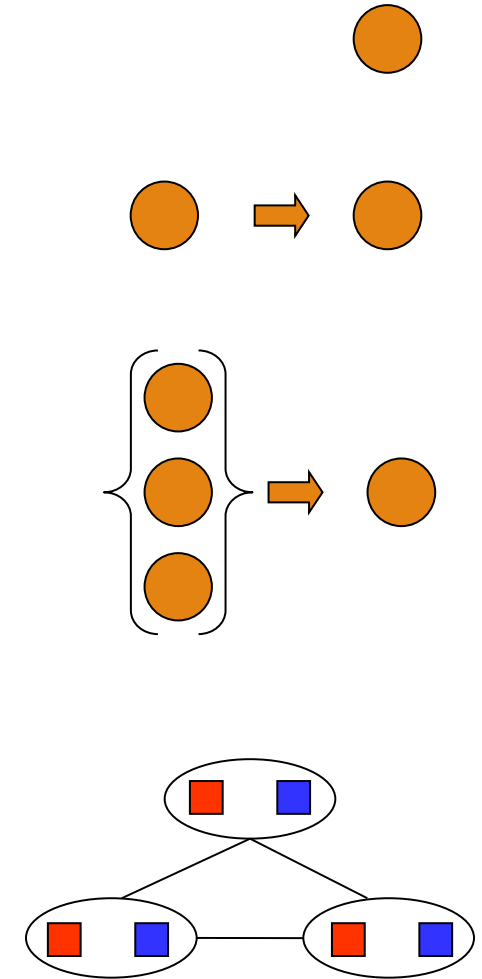
Limits of Arc Consistency

- After enforcing arc consistency:
 - Can have one solution left.
 - Can have multiple solutions left.
 - Can have no solutions left (and not detect it)
- In the last scenario, AC will not detect that no solution exists.
- Can we enforce a stronger notion of consistency?



K -consistency

- Increasing degrees of consistency
 - **1-Consistency (Node Consistency)**: Each single node's domain has a value which meets that node's unary constraints.
 - **2-Consistency (Arc Consistency)**: For each pair of nodes, any consistent assignment to one can be extended to the other.
 - **k -Consistency**: For any k nodes, any consistent assignment to $k - 1$ nodes can be extended to the k th node.
- Higher k more computation-expensive to enforce consistency.
- 3-consistency can be used to detect inconsistency in the coloring example shown here.



Backtracking search with inference

Pseudocode from book

INFERENCE can be replaced with forward-checking or AC-3, or any other consistency/filtering idea.

Note that this pseudocode did not explicitly make it clear to maintain the domains. So, one way to do this is to have a `domains_dictionary` in this recursive function.

Memory efficient implementation will maintain a single domain dictionary object for all recursive calls (pass-by-ref). Requires capability to *undo* domain changes when backtracking occurs; this requires more coding logic. (how would we do this?)

A memory inefficient solution is to deep copy domain dictionary in each recursive call, and pass as argument to next call. Any modification to deep copy is contained in each call. (why is this memory inefficient?)

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
    return BACKTRACK(csp, {})

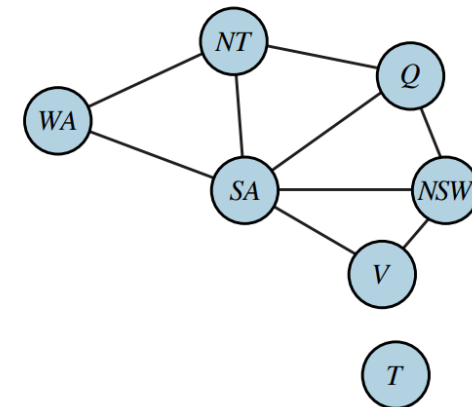
function BACKTRACK(csp, assignment) returns a solution or failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment)
    for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
        if value is consistent with assignment then
            add {var = value} to assignment
            inferences ← INFERENCE(csp, var, assignment)
            if inferences ≠ failure then
                add inferences to csp
                result ← BACKTRACK(csp, assignment)
                if result ≠ failure then return result
            remove inferences from csp
            remove {var = value} from assignment
    return failure
```

Figure 5.5 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. The functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES implement the general-purpose heuristics discussed in Section 5.3.1. The INFERENCE function can optionally impose arc-, path-, or *k*-consistency, as desired. If a value choice leads to failure (noticed either by INFERENCE or by BACKTRACK), then value assignments (including those made by INFERENCE) are retracted and a new value is tried.

Improving Backtracking Search – Exploiting problem structure

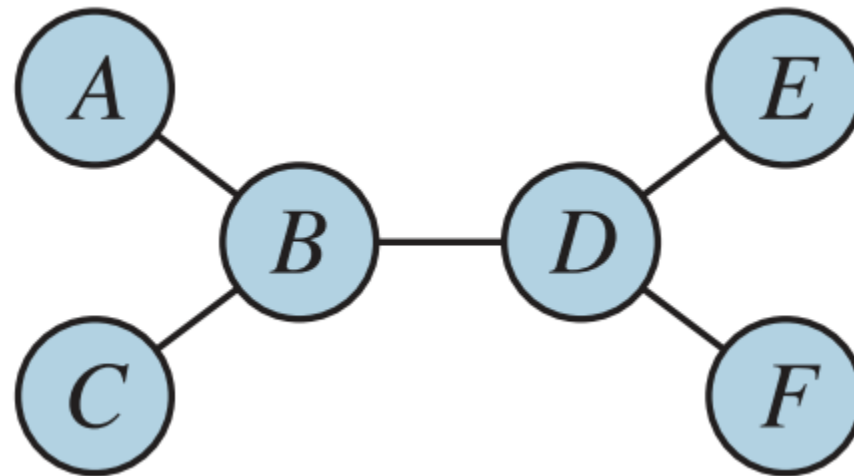
Problem Structure – idea 1 – independent subproblems

- Sometimes the structure of the CSP can be exploited to find a solution quickly.
- The problem can be decomposed into independent subproblems.
- Independent subproblems are identifiable as connected components of the constraint graph.
- For Australia map coloring, one component is T alone, another component is the subgraph WA, NT, SA, Q, NSW, V.
- Thus, the CSP = CSP₁ U CSP₂. Solution for each independently is solution to original CSP.
- Suppose a graph of n variables is decomposed into $\frac{n}{c}$ subproblems of c variables each:
 - Worst-case cost of backtracking is $O(\frac{n}{c} d^c)$ linear in n . Without decomposition $O(d^n)$ exponential in n .
 - E.g., $n = 80$, $d = 2$, $c = 20$; i.e. divide into 4 subproblems
 - $2^{80} = 4$ billion years at 1 million nodes/sec
 - $4 * 2^{20} = 0.4$ seconds at 1 million nodes/sec



Problem Structure – idea 2 – tree structured CSPs

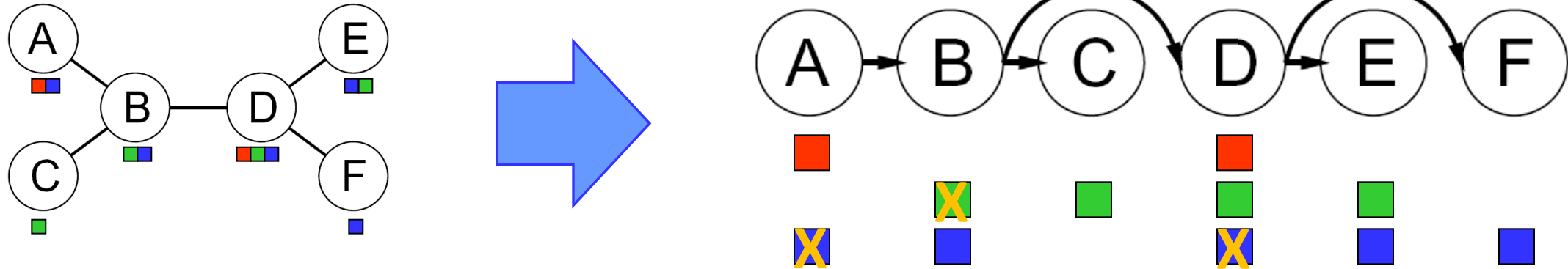
- If the constraint graph is tree-structured (i.e. no cycles), the CSP can be solved in $O(nd^2)$; linear in n .
- Compare to general CSPs, where worst-case time is $O(d^n)$



Problem Structure – tree structured CSPs

- Algorithm for tree-structured CSPs:

1. **Topological Sort:** Choose a root variable, order variables so that parents precede children



2. **Remove backward:** For $i = n$ to 2, enforce arc-consistency of $\text{Parent}(X_i) \rightarrow X_i$

3. **Assign forward:** For $i = 1$ to n , assign X_i consistently with $\text{Parent}(X_i)$

- Runtime: $O(nd^2)$. Why?

function TREE-CSP-SOLVER(*csp*) **returns** a solution, or *failure*

inputs: *csp*, a CSP with components X , D , C

$n \leftarrow$ number of variables in X

assignment $\leftarrow$ an empty assignment

root $\leftarrow$ any variable in X

$X \leftarrow$ TOPOLOGICALSORT(X , *root*)

for $j = n$ **down to** 2 **do**

 MAKE-ARC-CONSISTENT(PARENT(X_j), X_j)

if it cannot be made consistent **then return** *failure*

for $i = 1$ **to** n **do**

assignment[X_i] $\leftarrow$ any consistent value from D_i

if there is no consistent value **then return** *failure*

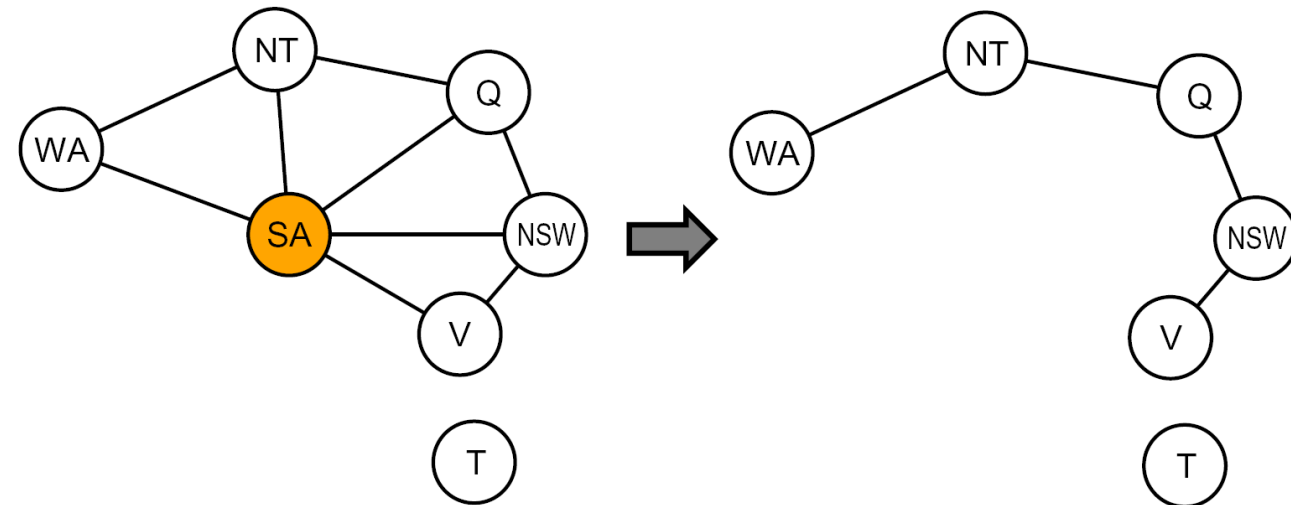
return *assignment*

← Notice here we are just making a single arc consistent; i.e. we are not running AC3 on all arcs in the CSP.

Figure 5.11 The TREE-CSP-SOLVER algorithm for solving tree-structured CSPs. If the CSP has a solution, we will find it in linear time; if not, we will detect a contradiction.

Cutset conditioning

- So, we have an efficient algorithm for solving tree-structured CSPs. How can we reduce general CSPs to tree-structured?
 - **Cutset conditioning**: removes nodes from constraint-graph.
 - **Tree-decomposition**: collapse nodes together.
- Cutset condition algorithm:
 1. Select a subset S of variables such that the CSP becomes a tree after the removal of the S nodes.
 2. For each consistent assignment of S :
 - a. Remove from domains of unassigned variables inconsistent values.
 - b. Apply tree-csp-algorithm on remaining variables. Return solution if successful.
- With cutset size c , runtime is $O(d^c (n - c) d^2)$, very fast for small c .



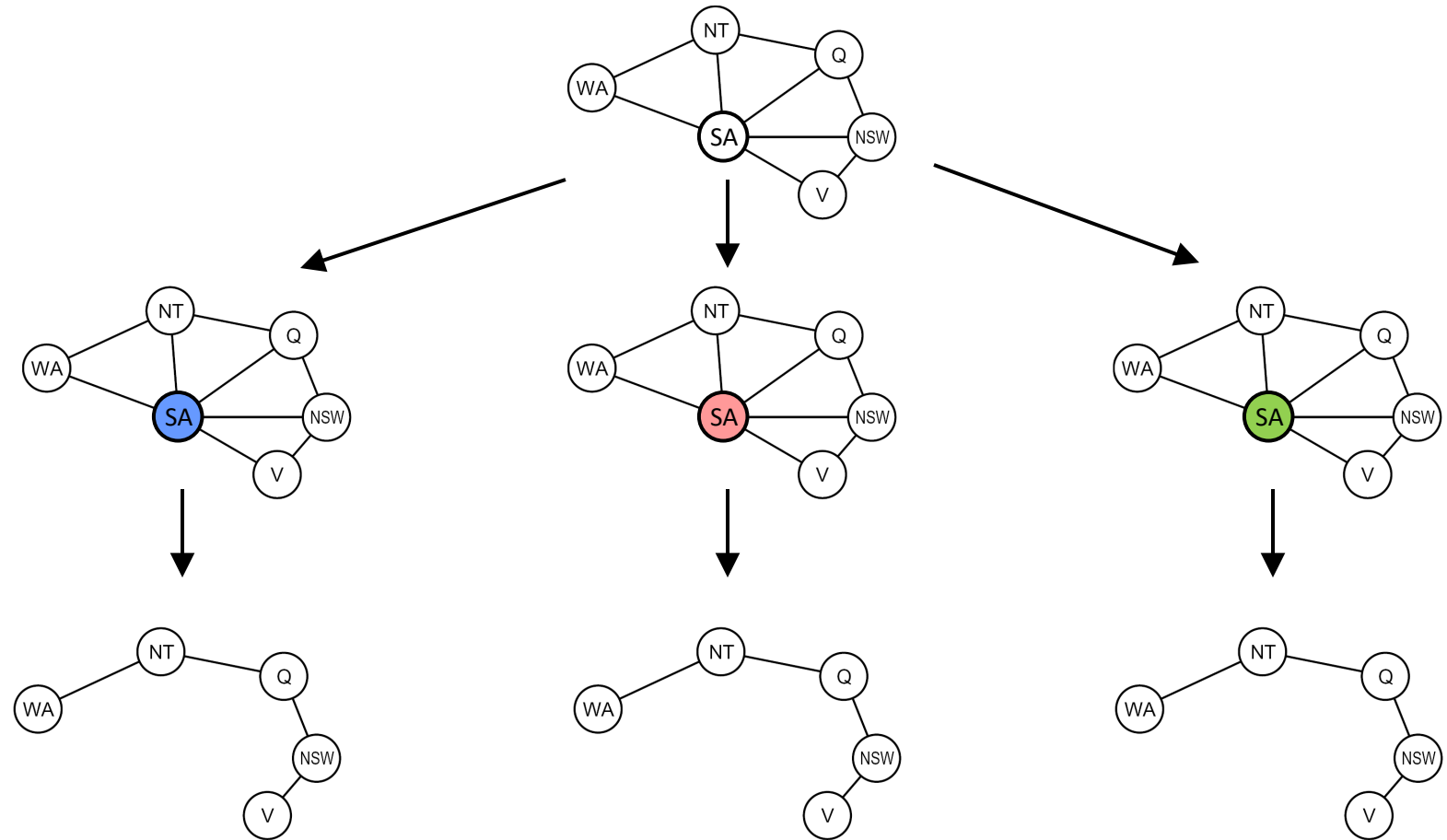
Cutset conditioning

Choose a cutset

Instantiate the cutset
(all possible ways)

Compute residual CSP
for each assignment

Solve the residual CSPs
(tree structured)



Local Search with CSPs

Local Search with CSPs

- So far, we have dealt with CSPs using partial assignments (partial state formulation).
- We can also consider **complete assignments (complete state formulation)**:
 - We did this in local search. A state is a complete configuration of the variables.
 - Initial state: random complete assignment that may or may not be consistent.
 - At each step: randomly modify one variable assignment.
- With complete assignments, we can apply local search methods to find a solution. Hill-climbing, simulated annealing, beam search, etc.
- Here is an idea for a CSP Local Search Algorithm:
 - For $i = 1, \dots, \text{max_steps}$:
 - **Variable selection**: randomly select any **conflicting** variable (check constraints).
 - **Value selection** using **min-conflicts** heuristic: choose the value that results in the min number of conflicts with other variables; if there are more than one with same number of conflicts, choose one at random.

CSP Local Search – 8-queens

- Given the current state, at each step:
 - A conflicting queen is randomly selected for reassignment in its column.
 - Move the queen to the min-conflicts square, breaking ties randomly.

An 8x8 chessboard illustrating the 8-queens problem. Each column contains exactly one queen. The number of conflicts for each column is shown in the rightmost cell of that column. Red arrows point to the top-right cells of the 6th and 7th columns, which both have a conflict count of 2.

				♔	3		2
♔					3		2
					♔		1
			♔		2		2
	♔				3		3
					2	♔	1
		♔			3		2
					0		♔

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or *failure*
 inputs: *csp*, a constraint satisfaction problem
 max_steps, the number of steps allowed before giving up

 current $\leftarrow$ an initial complete assignment for *csp*
 for *i* = 1 to *max_steps* **do**
 if *current* is a solution for *csp* **then return** *current*
 var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES
 value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*csp*, *var*, *v*, *current*)
 set *var* = *value* in *current*
 return *failure*

Figure 5.9 The MIN-CONFLICTS local search algorithm for CSPs. The initial state may be chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn. The CONFLICTS function counts the number of constraints violated by a particular value, given the rest of the current assignment.

CSP Local Search

- Can use Local search algorithms to solve CSPs.
- Local search may get stuck at local optima.
- Can use multiple initializations to improve, re-initialize randomly (“repeated” local search)
- Can also allow suboptimal moves based on probability (e.g. simulated annealing).
- Can also use simulated annealing, beam-search, genetic algorithm ideas with num-conflicts as objective function.
- On n-queens, runtime of min-conflicts is roughly independent of problem size.
 - Can solve the millions-queen problem in roughly 50 steps.

CSP comparisons

Problem	Backtracking	BT+MRV	Forward Checking	FC+MRV	Min-Conflicts
USA	(> 1,000K)	(> 1,000K)	2K	60	64
<i>n</i> -Queens	(> 40,000K)	13,500K	(> 40,000K)	817K	4K
Zebra	3,859K	1K	35K	0.5K	2K
Random 1	415K	3K	26K	2K	
Random 2	942K	27K	77K	15K	

- Numbers show median number of generated nodes over 5 runs to solve problem.
- Parentheses means no solution found and algorithm was stopped.
- USA: 4 coloring for 50 US states, n-queens: $n = 2$ to 50, Zebra problem see ex5.7, last two problems are synthetic random CSP problems.
- In practice, you want to run experiments on your problem to determine which method is best.

Summary

Summary

- CSPs are a special kind of search problem:
 - States are partial assignments
 - Goal test defined by constraints
- Solution methods:
 - Backtracking= DFS with one variable assigned per node
 - Local search
- Backtracking can be enhanced by
 - Variable/value ordering heuristics
 - FC to prevent assignments that result in later failure
 - Constraint propagation (e.g., arc consistency) does additional work to constrain values and detect inconsistencies
- Exploiting problem structure can speed up runtime of solving CSP. Particularly, reducing CSPs to tree-structured CSPs using cutset conditioning or tree-decomposition.